

COURSE

Основы программирования

JS

Объектная модель браузера (BOM)

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции *3*

Теоретическая часть *3*

Объект window **3**

Диалоговые окна 4

Методы объекта window: open(), close(), print(), focus() и blur(). 5

Свойства объекта window: name, opener, closed. 8

Методы для перемещения окна 10

Методы для изменения размеров окна 11

Методы, предназначенные для прокрутки документа внутри окна 12

Свойства innerWidth и innerHeight 13

Свойства outerWidth и outerHeight 14

Свойства screenLeft (screenX) и screenTop (screenY) 14

Свойства pageXOffset (scrollX) и pageYOffset (scrollX) 15

Объект screen **16**

Фреймы **17**

Объект history **22**

Объект location **25**

Объект navigator **28**

Объект geolocation **29**

Таймеры **30**

Функция setTimeout 30

Функция setInterval 31

requestAnimationFrame() 32

Литература и другие источники *33*

Вопросы для самопроверки *33*

ПЛАН ЛЕКЦИИ

1. Объект Window;
2. Объект Screen;
3. Фреймы;
4. Объект History;
5. Объект Location;
6. Объект Navigator;
7. Объект Geolocation;
8. Таймеры.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Веб-страницы бывают статическими и динамическими, последние отличаются тем, что в них используются сценарии (программы) на языке JavaScript.

В сценариях JavaScript браузер веб-разработчику предоставляет множество "готовых" объектов, с помощью которых он может взаимодействовать с элементами веб-страницы и самим браузером. В совокупности все эти объекты составляют ***объектную модель браузера*** (***BOM*** – Browser Object Model).

На самом верху этой модели находится глобальный объект window. Он представляет собой одно из окон или вкладку браузера с его панелями инструментов, меню, строкой состояния, HTML страницей и другими объектами. Доступ к этим различным объектам окна браузера осуществляется с помощью следующих основных объектов: navigator, history, location, screen, document и т.д. Так как данные объекты являются дочерними по отношению к объекту window, то обращение к ним происходит как к свойствам объекта window.

Из всех этих объектов, наибольший интерес и значимость для разработчика представляет объект document, который является корнем ***объектной модели документа*** (***DOM*** – Document Object Modal). Данная модель в отличие от объектной модели браузера стандартизована в спецификации и поддерживается всеми браузерами.

Объект document представляет собой HTML документ, загруженный в окно (вкладку) браузера. С помощью свойств и методов данного объекта Вы можете получить доступ к содержимому HTML-документа, а также изменить его содержимое, структуру и оформление.

Объект window.

Объект window представляет собой окно веб-браузера, в котором размещаются веб-страницы. window является глобальным объектом, поэтому при доступе к его свойствам и методам необязательно использовать его имя. Например, window имеет метод alert(), который отображает окно сообщения. Но нам необязательно писать:

```
window.alert("Привет мир!");
```

window можно не использовать:

```
alert("Привет мир!");
```

Но так как данный объект глобальный, то это накладывает некоторые ограничения. Например:

```
var alert = function(message){
    document.write("Сообщение: " + message);
};
```

```
window.alert("Привет мир!");
```

Все объявляемые в программе глобальные переменные или функции автоматически добавляются к объекту window. И поскольку название новой функции будет совпадать с названием метода alert(), то произойдет переопределение этого метода в объекте window новой функцией.

И если мы объявим в программе какую-нибудь глобальную переменную, то она нам доступна как свойство в объекте window:

```
var message = "hello";
document.write(window.message);
```

Диалоговые окна

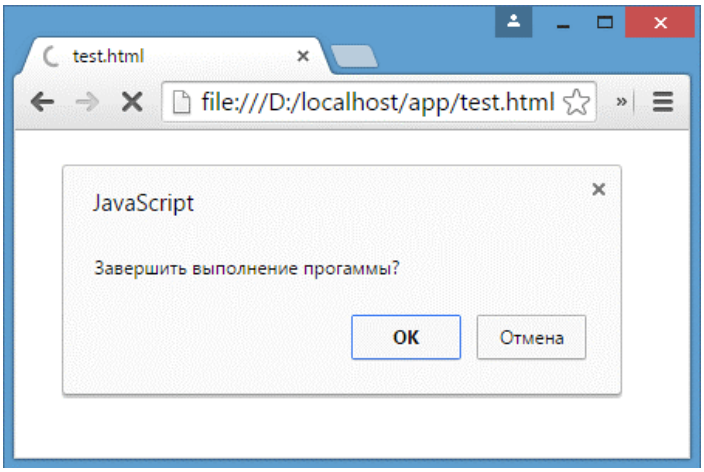
Для взаимодействия с пользователем в объекте window определен ряд методов, которые позволяют создавать диалоговые окна.

Метод ***alert()*** выводит окно с сообщением:

```
alert("hello world");
```

Метод ***confirm()*** отображает окно с сообщением, в котором пользователь должен подтвердить действие двух кнопок OK и Отмена. В зависимости от выбора пользователя метод возвращает true (если пользователь нажал OK) или false (если пользователь нажал кнопку Отмены):

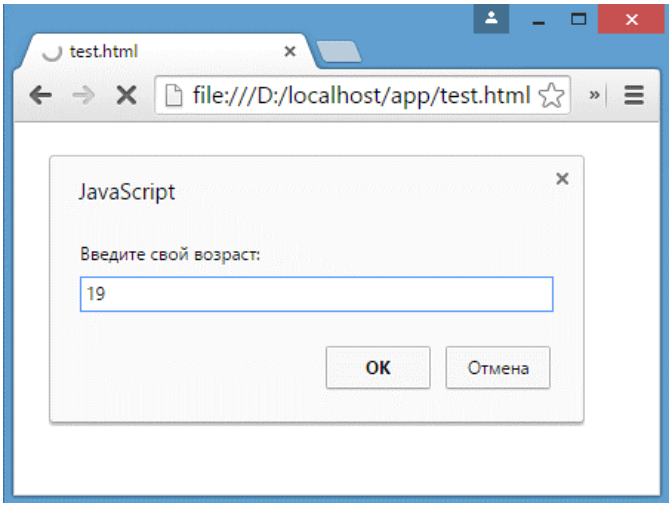
```
var result = confirm("Завершить выполнение программы?");
if(result===true)
    document.write("Работа программы завершена");
else
    document.write("Программа продолжает работать");
```



И метод ***prompt()*** позволяет с помощью диалогового окна запрашивать у пользователя какие-либо данные. Данный метод возвращает введенное пользователем значение:

```
var age = prompt("Введите свой возраст:");
document.write("Вам " + age + " лет");
```

Если пользователь откажется вводить значение и нажмет на кнопку отмены, то метод возвратит значение null.



Методы объекта window: open(), close(), print(), focus() и blur().

В этом разделе мы рассмотрим следующие методы объекта window:

- **open()** - предназначен для открытия окон (вкладок);
- **close()** - предназначен для закрытия окон. В основном используется для закрытия окон, открытых методом open();
- **print()** - предназначен для печати содержимого окна;
- **focus()** - предназначен для передачи фокусу указанному окну;
- **blur()** - предназначен для удаления фокуса с указанного окна.

Метод ***open()*** предназначен для открытия нового окна (вкладки) в браузере и имеет следующий синтаксис:



В первом параметре указывается URL страницы, которую необходимо загрузить в это окно. Если значение данного параметра не задавать, то в окне будет отображена пустая страница (about:blank).

Во втором параметре метода open указывается значение атрибута target или имя окна. Поддерживаются следующие значения:

- **_blank** - URL загружается в новое окно (вкладку). Это значение используется по умолчанию;
- **_parent** - URL загружается в родительский фрейм. Если его нет, то URL загружается в текущее окно (вкладку);
- **_self** - URL загружается в текущее окно;
- **_top** - отменяет все фреймы и загружает URL в текущее окно (вкладку) браузера. Если его нет, то URL загружается в текущее окно (вкладку);

В качестве параметра можно указать имя открываемого окна. Данное имя является внутренним и может использоваться веб-разработчиками для вызова функций и методов этого окна.

Третий параметр предназначен для указания набора свойств окна, которые вводятся через запятую. Поддерживаются следующие основные свойства окна:

- **left, top** - координаты (в пикселях) левого верхнего угла окна браузера относительно левого верхнего угла экрана. Значения этих свойств должны быть положительными или равными 0;
- **height, width** — высота и ширина рабочей области окна браузера. При указании значений необходимо учитывать, что ширина и высота окна браузера не может быть меньше 100 пикселей;
- **resizable** — логическое свойство окна, которое предназначено для включения или выключения возможности изменения размеров окна браузера. Данное свойство принимает следующие значения: yes или 1, и no или 0;
- **scrollbars** - логическое свойство окна, которое предназначено для включения или выключения отображение полос прокрутки для содержимого окна браузера. Данное свойство принимает следующие значения: yes или 1, и no или 0;
- **status** - логическое свойство окна, которое предназначено для включения или выключения отображения статусной строки браузера. Данное свойство принимает следующие значения: yes или 1, и no или 0.

Рассмотрим следующий примеры:

Открыть пустую страницу about:blank в новом окне. Данное окно должно иметь ширину и высоту, равную 250px:

```
window.open("","","width=250, height=250");
```

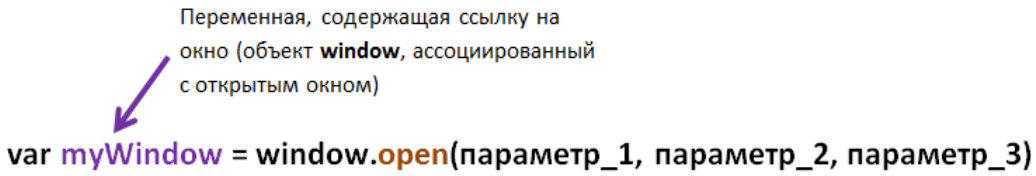
Открыть веб-страницу "http://itchief.ru/" в текущем окне:

```
window.open("http://itchief.ru/", "_self");
```

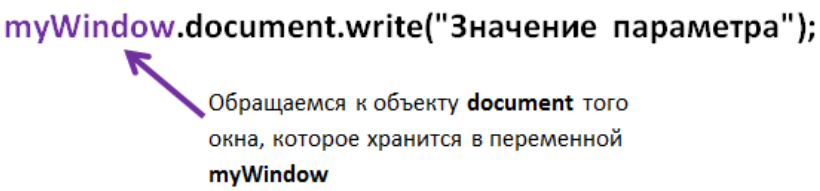
Открыть новое окно, имеющее определённые свойства (top=100, left=100, width=400, height=500, scrollbars=yes, resizabie=yes):

```
window.open("http://itchief.ru", "_blank", "top=100, left=100, width=400, height=500, scrollbars=yes, resizable=yes");
```

Метод ***open()*** позволяет не только открыть окно, но и получить ссылку на данное окно. Данная ссылка позволяет взаимодействовать с этим окном посредством вызова определённых свойств и методов. Т.е. мы можем с помощью JavaScript кода, расположенного в одном окне управлять другим окном.



Например, для того чтобы обратиться к объекту document открытого окна:



Открыть пустое новое окно и вывести в ней некоторый текст:

```
var myWindow = window.open("", "", "width=250, height=250");
myWindow.document.write("<p>Некоторый текст</p>");
```

Примечание: Взаимодействовать Вы можете только с теми окнами, которые сами открыли, с другими окнами Вы работать не можете.

Метод ***close()*** предназначен для закрытия окна. Данный метод не имеет параметров. Он обычно используется для закрытия окон, созданных методом open(). В противном случае, когда Вы попытаетесь закрыть окно (вкладку), открытое самим пользователем (не из JavaScript), то браузер из-за соображений безопасности запросит у пользователя подтверждение на выполнение этого действия.

Например, создадим кнопки для открытия и закрытия окна с именем myWindow:

```
<script>
//создать переменную, в которой будем хранить ссылку на объект window открытого окна
var myWindow;
function myWindowOpen {
  myWindow = window.open("http://www.yandex.ru", “myWindow", "width=250, height=250");
}
function myWindowClose {
  if (myWindow) {
    myWindow.close();
    myWindow = null;
  }
}
</script>

<button onClick="myWindowOpen()">Открыть окно</button>
<button onClick="myWindowClose()">Закрыть окно</button>
```

Метод ***print()*** предназначен для печати содержимого окна. Данный метод не имеет параметров.

```
<script>
function myPrint() {
  window.print();
}
</script>
<button onclick="myPrint()">Печать страницы</button>
```

Метод ***focus()*** предназначен для передачи фокусу указанному окну. Данный метод не имеет параметров.

Метод ***blur()*** предназначен, чтобы убрать фокус с указанного окна, т.е. перемещает его на задний план. Данный метод не имеет параметров.

```
<script>
function myWindowOpen() {
  var myWindow = window.open("", "", "width=200, height=100");
}
function myWindowFocus() {
  myWindow.focus();
}
```

```
function myWindowBlur() {
  myWindow.blur();
}
</script>

<button onclick="myWindowOpen()">Открыть окно</button>
<button onclick="myWindowFocus()">Передать фокус окну</button>
<button onclick="myWindowBlur()">Переместить окно на задний план</button>
```

Свойства объекта window: name, opener, closed.

В этом разделе рассмотрим следующие свойства объекта window:

- **name** — предназначено для получения или установления внутреннего имени окна;
- **opener** - позволяет получить в текущем окне, ссылку на окно (объект window), с которого было открыто данное окно;
- **closed** - свойство логического типа, которое возвращает: true, если окно закрыто и false, если окно открыто.

Свойство name очень часто используется для изменения внутреннего имени окна, после того как оно уже открыто. Кроме этого, свойство name может вернуть текущее значение внутреннего имени окна.

Внутреннее имя окна, это не строка, заключённая между открывающим и закрывающим тегом title - это имя окна которое предназначено для разработчика. Т.е. данное имя невидимо для пользователя.

Данное имя в основном используется в гиперссылках и формах для указания окна, в котором необходимо открыть страницу. Например, для указания внутреннего имя окна в гиперссылке используется атрибут target. Если элемент a имеет атрибут target="searchWindow", то при нажатии на данную ссылку браузер сначала пытается найти окно с таким внутренним именем (searchWindow), если окна с таким внутренним именем не существует, то он открывает новое окно и присваивает ему имя searchWindow. А если окно с таким именем существует, то новое окно не открывается, а перезагружается страница по указанной ссылке в этом окне. По умолчанию окна в браузере не имеют внутреннего имени.

Например, откроем страницу "http://www.google.com/" в окне, имеющем имя myWindow:

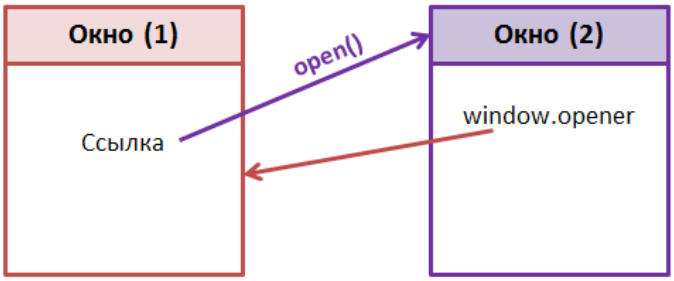
```
<!-- Изменим имя текущего окна на "myWindow" -->
window.name = "myWindow";
<!-- Откроем URL, указанный в атрибуте href, в окне, имеющей имя "myWindow" -->
<a href="http://www.google.com/" target="myWindow"></a>
```

Например, откроем окно с помощью метода open() и выведем в нём его имя:

```
<!-- Откроем окно и зададим ему внутреннее имя "myTest" -->
var wnd = window.open("", "myTest", "width=200, height=300");
<!-- выведем с помощью метода write объекта document внутреннее имя окна -->
wnd.document.write("<p>Это окно имеет имя:" + wnd.name +".</p>");
```

Свойство opener позволяет получить в окне, ссылку на исходное окно (объект window), т.е. на окно из которого было открыто данное окно.

Например, у Вас есть исходное окно (1), в котором Вы с помощью метода open() открываете другое окно (2). В этом окне (2) Вы можете с помощью свойства opener получить окно (1).




```
<script>
function openMyWindow(){
  <!-- Открываем пустое окно, имеющее ширину и высоту, равные 200px -->
  var myWindow=window.open("", "", "width=200, height=200");
  <!--– Выводим в открывшееся окно заголовок h1 -->
  myWindow.document.write("<h1>Открытое окно (2)</h1>");
  <!-- В окне myWindow (2), с помощью свойства opener получаем исходное окно (1), в котором посредством метода write объекта document выводим заголовок h1 –>
  myWindow.opener.document.write("<hl>Это исходное окно(1), с помощью которого мы открыли окно (2)</h1>");
}
</script>

<a href="javascript:openMyWindow()">Открыть окно</a>
```

Свойство closed возвращает логическое значение, указывающее закрыто окно или нет.

Пример:

```
<button onclick="openWindow()">Открыть окно</button>
<button onclick="closeWindow()">Закрыть окно</button>
<button onclick="stateWindow()">Состояние окна</button>

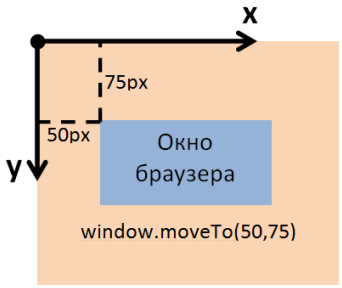
<script>
var myTestWindow;
function openWindow() {
  myTestWindow = window.open ("", "", "left=200, top=250, width=250, height=250");
}
function closeWindow() {
  if (myTestWindow) {
    myTestWindow.close();
  }
}
function stateWindow() {
  if (!myTestWindow) {
    alert("Окно не открыто");
  }
  else {
    if (myTestWindow.closed)
      alert ("Окно закрыто");
    else
      alert ("Окно открыто");
  }
```

```
}
</script>
```

Методы для перемещения окна

В JavaScript для перемещения окна доступны следующие методы:

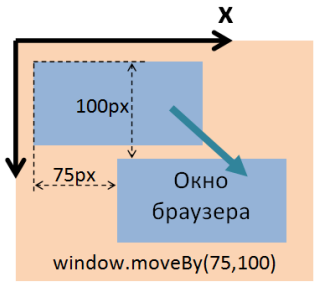
- **moveTo()** - предназначен для перемещения левого верхнего угла окна в указанную точку экрана;
- **moveBy()** - предназначен для перемещения окна на указанное количество пикселей относительно его текущего положения.



Метод ***moveTo()*** имеет синтаксис : window.moveTo(x, y);

Параметры метода:

- **x** - число (количество пикселей), которое задаёт горизонтальную координату левого верхнего угла окна браузера относительно верхнего левого угла экрана дисплея;
- **y** – число (количество пикселей), которое задаёт вертикальную координату левого верхнего угла окна браузера относительно верхнего левого угла экрана дисплея.



Метод ***moveBy()*** имеет синтаксис: window.moveBy(x, y);

Параметры метода:

- **x** - положительное или отрицательное целое число, которое указывает количество пикселей, на которые необходимо переместить окно браузера относительно его текущего положения в горизонтальном направлении;
- **y** - положительное или отрицательное целое число, которое указывает количество пикселей, на которые необходимо переместить окно браузера относительно его текущего положения в вертикальном направлении.

Пример:

```
<script>
var myWindow;
function openWindow() {
  myWindow = window.open("", "", "width=200, height=200");
```

```
  }
  function moveWindowTo() {
    if (myWindow && !myWindow.closed) {
      myWindow.moveTo(300, 300);
       myWindow.focus();
    }
  }
  function moveWindowBy(x, y) {
    if (myWindow && !myWindow.closed) {
      myWindow.moveBy(x, y);
       myWindow.focus();
    }
  }
  </script>
  ...
  <button onclick="openWindow()">Открыть окно</button>
  <button onclick="moveWindowTo()">Переместить левый угол окна в точку (300,300)</button>
  <button onclick="moveWindowBy(50,50)">Переместить окно относительно его текущего положения на 50px вправо и на 50px вниз</button>
  <button onclick="moveWindowBy(-50,0)">Переместить окно относительно его текущего положения на 50px влево</button>
  <button onclick="moveWindowBy(50,0)">Переместить окно относительно его текущего положения на 50px вправо</button>
  <button onclick="moveWindowBy(0,-50)">Переместить окно относительно его текущего положения на 50px вверх</button>
  <button onclick="moveWindowBy(0,50)">Переместить окно относительно его текущего положения на 50px вниз</button>
```

Методы для изменения размеров окна

В JavaScript доступны для изменения размеров окна следующие методы:

- **resizeTo()** - изменяет ширину и высоту окна на умазанные значения, заданные с помощью соответствующих значений параметров;
- **resizeBy()** - предназначен для изменения окна на указанное количество пикселей относительно его текущего размера.

Эти методы изменяют размеры окна посредством перемещения правого нижнего угла окна браузера, при этом верхний левый угол окна не перемещается (он остаётся в своих прежних координатах).

Метод ***resizeTo()*** имеет синтаксис: window.resizeTo(width,height);

Параметры метода:

- **width** - устанавливает ширину окна браузера в пикселях;
- **height** - устанавливает высоту окна браузера в пикселях.

Метод ***resizeBy()*** имеет следующий синтаксис: window.resizeBy(width,height);

Параметры метода:

- **width** - увеличивает или уменьшает ширину окна браузера на указанное количество пикселей;
- **height** - увеличивает или уменьшает высоту окна браузера на указанное количество пикселей.

Пример:

```
<script>
var myWindow;
function openWindow() {
  myWindow = window.open("", "", "width=200, height=200");
}
function resizeWindowTo() {
  if (myWindow && !myWindow.closed) {
    myWindow.resizeTo(300, 300);
     myWindow.focus();
  }
}
function resizeWindowBy(x, y) {
  if (myWindow && !myWindow.closed) {
    myWindow.resizeBy(x, y);
     myWindow.focus();
  }
}
</script>
...
<button onclick="openWindow()">Открыть окно</button>
<button onclick="resizeWindowTo()">Изменить ширину и высоту окна браузера (300,300)</button>
<button onclick="resizeWindowBy(25, 25)">Увеличить ширину и высоту окна браузера на 25px</button>
<button onclick="resizeWindowBy(-25, 0)">Уменьшить ширину окна на 25px</button>
<button onclick="resizeWindowBy(25, 0)"> Увеличить ширину окна на 25px</button>
<button onclick="resizeWindowBy(0, -25)"> Уменьшить высоту окна на 25px</button>
<button onclick="resizeWindowBy(0, 25)"> Увеличить высоту окна на 25px</button>
```

Методы, предназначенные для прокрутки документа внутри окна

В JavaScript для прокрутки HTML документа внутри окна доступны следующие методы:

- **scrollTo()** - предназначен для прокрутки документа в горизонтальном и вертикальном направлении на указанное количество пикселей;
- **scrollBy()** — предназначен для прокрутки документа вверх или вниз и влево или вправо на определённое количество пикселей относительно его текущего положения.

Метод ***scrollTo()*** имеет следующий синтаксис: window.scrollTo(x, y);

Параметры метода:

- **x** - координата вдоль оси X, до которой необходимо прокрутить документ в горизонтальном направлении;
- **y** - координата вдоль оси Y, до которой необходимо прокрутить документ в вертикальном направлении.

Метод ***scrollBy()*** имеет следующий синтаксис: window.scrollBy(x, y);

Параметры метода:

- **x** - указывает количество пикселей, на которое необходимо прокрутить документ в горизонтальном направлении относительно его текущего положения. Положительное значение данного параметра прокручивает документ вправо, в то время как отрицательное значение влево;
- **y** - указывает количество пикселей, на которое необходимо прокрутить документ в вертикальном направлении относительно его текущего положения. Положительное значение данного параметра прокручивает документ вниз, в то время как отрицательное значение вверх.

Пример:

```
<script>
function scrollWindowTo() {
  window.scrollTo(200, 300);
}
function scrollWindowBy(x, y) {
  window.scrollBy(x, y);
}
</script>
...
<button onclick="scrollWindowTo()">Прокрутить документ по X и Y на 200px и 300px соответственно</button>
<button onclick="scrollWindowBy(100, 0)">Прокрутить документ на 100px вправо относительно его текущего положения</button>
<button onclick="scrollWindowBy(-100, 0)"> Прокрутить документ на 100px влево относительно его текущего положения </button>
<button onclick="scrollWindowBy(0, 50)"> Прокрутить документ на 50px вниз относительно его текущего положения </button>
<button onclick="scrollWindowBy(0, -50)"> Прокрутить документ на 50px вверх относительно его текущего положения </button>
```

Свойства innerWidth и innerHeight

Они предназначены для получения размеров видимой рабочей области окна браузера. Т.е. свойства innerWidth и innerHeight предназначены для получения значений ширины и высоты области, в которой отображается HTML-документ. Эти свойства доступны только для чтения и возвращают значения в пикселях.

Например, получить высоту и ширину видимой рабочей области окна браузера:

```
<p>Ширина видимой области просмотра (widthContenArea): <span id="widthContenArea"></span></p>
<p>Ширина видимой области просмотра (heightContenArea): <span id="heightContenArea"></span></p>
<script>
// ширина видимой области просмотра (для всех браузеров, кроме Internet Explorer 8)
var widthContenArea = window.innerWidth;
// высота видимой области просмотра (для всех браузеров, кроме Internet Explorer 8)
var heightContenArea = window.innerHeight;
// ширина видимой области просмотра (для Internet Explorer 8)
widthContenArea = document.documentElement.clientWidth || document.body.clientWidth;
// высота видимой области просмотра (для Internet Explorer 8)
heightContenArea = document.documentElement.clientHeight || document.body.clientHeight;
```

```
// ширина видимой области просмотра (для всех браузеров)
widthContenArea1 = window.innerWidth || document.documentElement.clientWidth ||
document.body.clientWidth;
// высота видимой области просмотра (для всех браузеров)
heightContenArea1 = window.innerHeight || document.documentElement.clientHeight ||
document.body.clientHeight;
document.getElementById("widthContenArea").innerHTML = widthContenArea;
document.getElementById("heightContenArea").innerHTML = heightContenArea;
</script>
```

Свойства outerWidth и outerHeight

Они предназначены для получения размеров всего окна браузера, т.е. включая панели инструментов, полосы прокрутки, строку состояния, границы окна и т.д. Свойства outerWidth и outerHeight доступны только для чтения и возвращают соответственно ширину и высоту окна в пикселях.

Например, получить высоту и ширину окна браузера:

```
<p>Ширина окна браузера (widthWindow): <span id="widthWindow"></span></p>
<p>Высота окна браузера (heighWindow): <span id="heightWindow"></span></p>
<script>
// ширина окна браузера
var widthWindow = window.outerWidth;
// высота окна браузера
var heightWindow = window.outerHeight;
document.getElementById("widthWindow").innerHTML = widthWindow;
document.getElementById("heightWindow").innerHTML = heightWindow;
</script>
```

Свойства screenLeft (screenX) и screenTop (screenY)

Они предназначены для получения координаты левого верхнего угла окна браузера или документа относительно левого верхнего угла экрана пользователя.

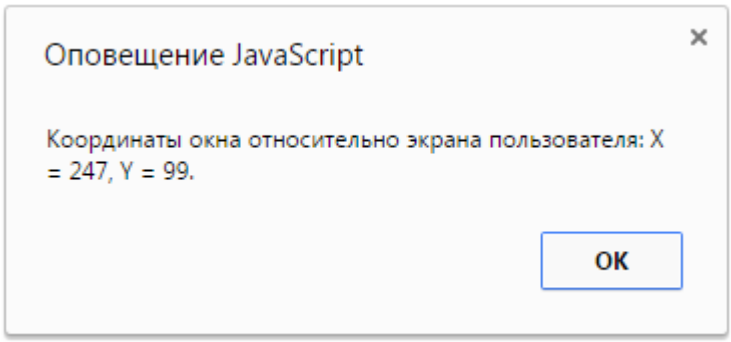
При этом свойства screenLeft и screenTop работают в Internet Explorer, а свойства screenX и screenY в Mozilia Firefox. В браузерах Chrome, Safari и других подобных браузерах можно использовать как свойства screenLeft и screenTop, так и свойства screenX и screenY.

При использовании данных свойств необходимо учитывать тот факт, что некоторые браузеры могут возвращать координату левого верхнего угла документа, а некоторые браузеры координату левого верхнего угла окна. Свойства screenleft (screenX) и screenTop (screenY) доступны только для чтения и возвращают соответственно значения расстояний относительно левого угла экрана по горизонтали и вертикали в пикселях.

Например, выведем виде сообщения координаты x и y левого угла текущего окна браузера (документа) относительно левого верхнего угла экрана:

```
<script>
function windowXY() {
  // Используя свойства screenleft и screenTop, получаем координаты расположения окна
(документа)
  var winX = window.screenLeft;
  var winY = window.screenTop;
  // Используя свойства screenX и screenY, получаем координаты расположения окна (до-
кумента)
```

```
    winX = window.screenX;
    winY = window.screenY;
    // Получаем координаты расположения окна (документа) во всех браузерах
    winX = window.screenX ? window.screenX : window.screenLeft;
    winY = window.screenY ? window.screenY : window.screenTop;
    window.alert ("Координаты окна относительно экрана пользователя: X = " + winX + ", Y
= " + winY + ".");
  }
  </script>
  <a href="javascript:windowXY()">Узнать координаты</a>
```

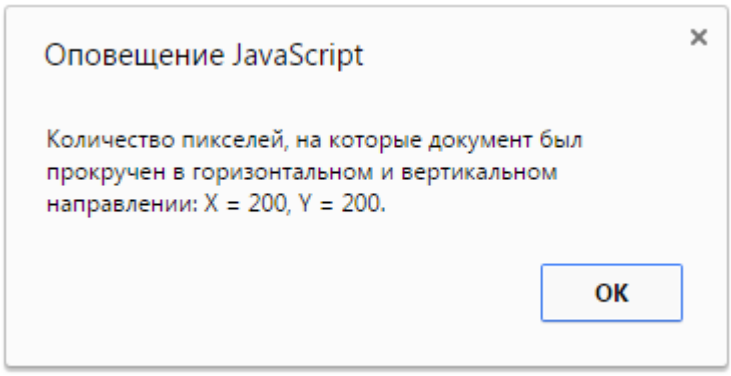


Свойства pageXOffset (scrollX) и pageYOffset (scrollX)

Они предназначены для получения количества пикселей, на которые документ был прокручен в горизонтальном (pageXOffset) или вертикальном (pageYOffset) направлении относительного левого верхнего угла окна. Свойства scrollX и scrollY эквиваленты соответственно свойствам pageXOffset и pageYOffset. Эти свойства доступны только для чтения.

Например, вывести в сообщении количество пикселей, на которые документ был прокручен в горизонтальном (pageXOffset) и вертикальном (pageYOffset) направлении.

```
  <script>
  function scrollContent() {
    //Прокрутим текущий документ на 200px вправо и вниз
    window.scrollBy(200,200);
    //Получим значения, используя свойства pageXOffset и pageYOffset
    var winOffsetX = window.pageXOffset;
    var winOffsetY = window.pageYOffset;
    //Получим значения, на которые документ был прокручен в горизонтальном или вертикальном направлении для Internet Explorer:
    winOffsetX = document.documentElement.scrollLeft;
    winOffsetY = document.documentElement.scrollTop;
    //Для всех браузеров:
    winOffsetX = window.pageXOffset || document.documentElement.scrollLeft;
    winOffsetY = window.pageYOffset || document.documentElement.scrollTop;
    alert ("Количество пикселей, на которые документ был прокручен в горизонтальном и вертикальном направлении: X = " + winOffsetX + ", Y = " + winOffsetY + ".");
  }
  </script>
  <a href="javascript:scrollContent()">Узнать положения полос прокрутки</a>
```



Объект screen

Объект screen - это один из дочерних объектов window, который может предоставить некоторую информацию об экране пользователя. Доступ к данному объекту осуществляется как к свойству объекта window, т.е. через точку.

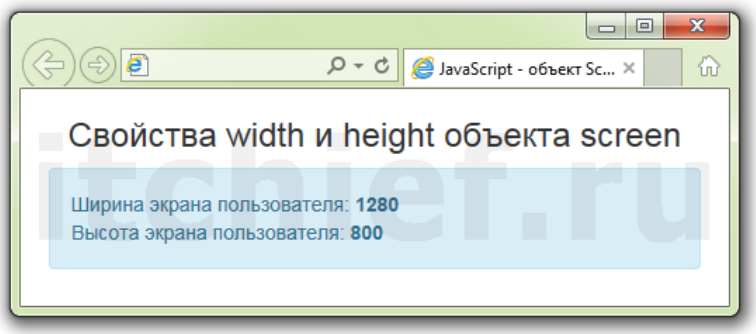
```
window.screen
```

Для получения различных характеристик экрана пользователя объект screen предоставляет для разработчика JavaScript следующие свойства: screen.width, screen.height, screen.availWidth, screen.availHeight, screen.colorDepth.

Свойства ***screen.width*** и ***screen.height*** возвращают соответственно ширину и высоту экрана пользователя в пикселях.

Например, вывести в элемент div ширину и высоту экрана пользователя:

```
<div id="userScreen" class="alert alert-warning"></div>
<script>
var userScreen;
userScreen = "Ширина экрана пользователя: <strong>" + screen.width + "</strong>";
userScreen += "<br />";
userScreen += "Высота экрана пользователя: <strong>" + screen.height + "</strong>";
window.document.getElementById("userScreen").innerHTML = userScreen;
</script>
```

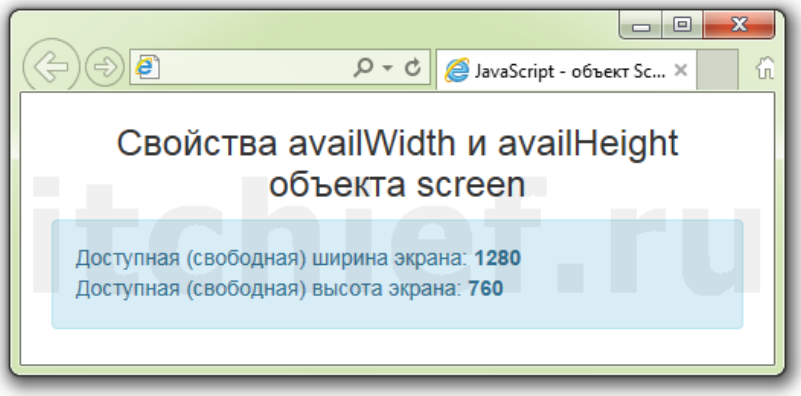


Свойства ***screen.availWidth*** и ***screen.availHeight*** возвращают ширину и высоту, которое может занять окно браузера. Например, в операционной системе Windows свойство screen.availHeight равно разнице между высотой экрана и высотой "Панели задач".

Например, вывести в элемент div доступную ширину и высоту экрана пользователя:

```
<div id="availSizeWindow" class="alert alert-warning"></div>
<script>
```

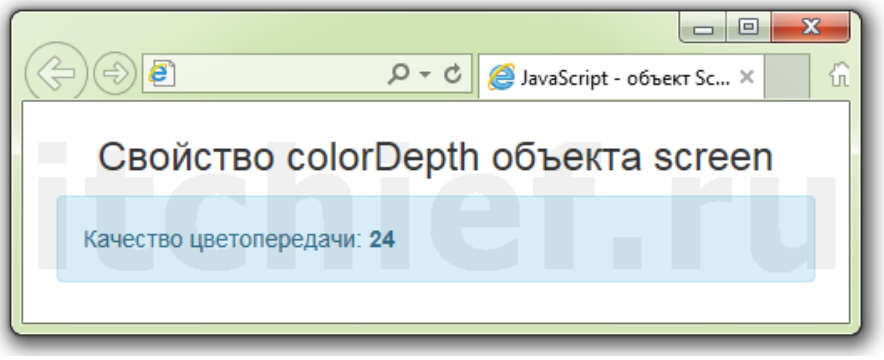
```
    var availSizeWindow;
    availSizeWindow = "Доступная (свободная) ширина экрана: <strong>" + screen.availWidth
+ "</strong>";
    availSizeWindow += "<br />";
    availSizeWindow += " Доступная (свободная) высота экрана: <strong>" + screen.avail-
Height + "</strong>";
    window.document.getElementById("availSizeWindow").innerHTML = availSizeWindow;
    </script>
```



Свойство ***screen.colorDepth*** возвращает количество бит, которые требуются для отображения одного цвета.

Например, вывести в элемент div качество цветопередачи устройства пользователя:

```
<div id="colorDepth" class="alert alert-warning"></div>
<script>
var colorDepth;
colorDepth = "Качество цветопередачи: <strong>" + screen.colorDepth + "</strong>";
window.document.getElementById("colorDepth").innerHTML = colorDepth;
</script>
```



Фреймы

Фреймы - это элементы HTML, которые используются для внедрения в текущую страницу других веб-страниц. Фреймы в HTML5 представлены с помощью элемента iframe (<iframe></iframe>). Для указания URL, который будет отображаться внутри фрейма (iframe) используется атрибут src.

Например, внутри фрейма отобразим страницу "http://getbootstrap.com/".

```
<iframe src="http://getbootstrap.com/"></iframe>
```

Фреймы немного похожи с картинками, только вместо картинки отображается внешний ресурс (HTML страница). Размещать фреймы (iframe) можно в любом месте страницы. Они относятся к строчным элементам. Кроме атрибута src у тега iframe есть множество других атрибутов: width (ширина), height (высота), name (имя) и др.

Например, создадим веб-страницу, содержащую ссылку и фрейм с именем. При нажатии на ссылку будем открывать в этом фрейме указанную веб-страницу.

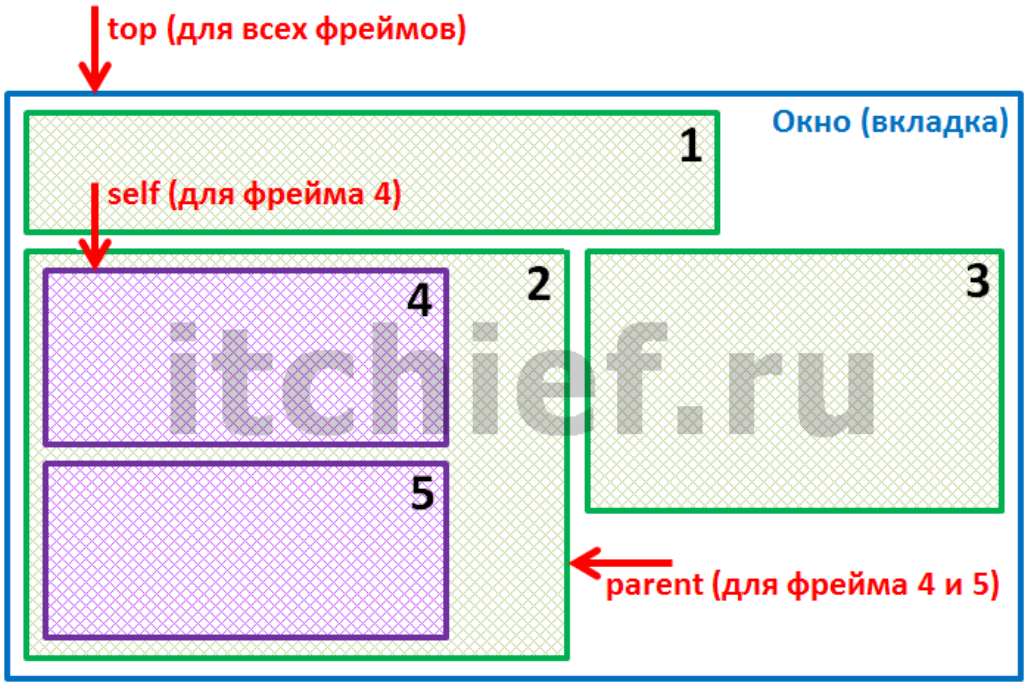
```
<a href="http://getbootstrap.com/" target="iframeBootstrap">Открыть ссылку во фрейме</a>
<iframe name="iframeBootstrap"></iframe>
```

С точки зрения объектной модели браузера фреймы (iframe) - это тоже окна (объекты window), которые можно расположить в основном окне (вкладке) браузера или в другом фрейме.

Свойства объекта window для работы с фреймами

Для работы с фреймами объект window содержит следующие свойства: frameElement, frames, length, self, parent и top. Разберём назначение этих свойств на примере.

Рассмотрим окно, состоящее из 5 фреймов (<iframe>...</iframe>). Первые три фрейма (1,2 и 3) расположены непосредственно в этом окне, а остальные 2 фрейма (4 и 5) расположены во фрейме 2.



Для фрейма 4:

- свойство parent, возвращает родительское окно (объект window), т.е. фрейм 2.
- свойство top, возвращает прародителя (предка), т.е. основное окно (вкладку), куда загружены все фреймы.
- свойство self, возвращает текущий объект window, т.е. фрейм 4.

Для фрейма 2:

- свойство parent, возвращает родительское окно (объект window), т.е. основное окно (вкладку).

- свойство top, возвращает прародителя (предка), т.е. основное окно (вкладку), куда загружены все фреймы.
- свойство self, возвращает текущий объект window, т.е. фрейм 2.

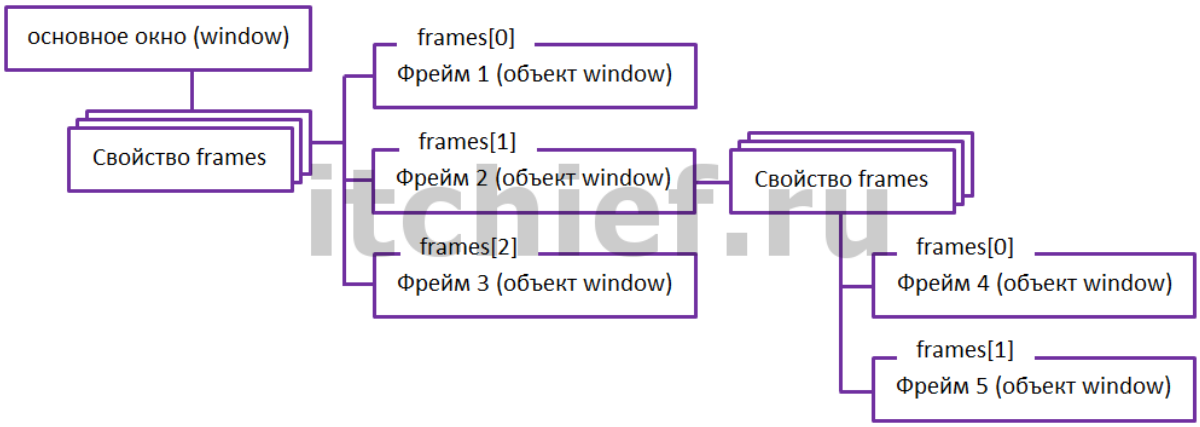
Свойство ***frameElement*** возвращает сам элемент iframe, или null если он таковым не является (например, если он является обычным окном, а не фреймом).

Свойство ***length*** обычно используется совместно со свойством frames и возвращает количество фреймов, содержащихся в текущем окне или фрейме. Например, в нашем примере свойство length для окна 2, вернёт значение 2. А для основного окна, свойство length вернёт значение 3. Свойство frames возвращает массив объектов window, каждый из которых отвечает за вложенный в это окно фрейм (iframe). Фреймы с логической точки зрения тоже являются окошечками, т.е. для каждого фрейма создаётся свой собственный объект window.

Вернёмся к нашему примеру, в котором основное окно (вкладка) браузера содержит 3 фрейма. В этом месте возникает вопрос: "А сколько всего объектов window будет создано?". В нашем случае будет создано 4 объекта window. Первый объект window будет отвечать за основное окно (вкладку) браузера. Три остальных объекта window будут отвечать за каждый из фреймов, непосредственно расположенных в этом окне. Эти объекты window, т.е. те которые отвечают за каждый фрейм, образуют массив frames, который можно получить как свойство объекта window.

Например:

```
<!--1 фрейм-->
window.frames[0];
<!--2 фрейм-->
window.frames[1];
<!--3 фрейм-->
window.frames[2];
```



itchief.ru

Свойство frameElement возвращает элемент iframe, если текущее окно является фреймом. В противном случае данное свойство, возвращает значение null (т.е. данное окно не является фреймом).

Например, узнать является ли текущее окно фреймом (iframe) и если является, то изменить его URL на "http://getbootstrap.com/":

```
//получить элемент iframe или значение null
var iframe = window.frameElement;
//Если окно является фреймом (iframe), то изменить URL фрейма на "http://getbootstrap.com/"
if (iframe) {
  iframe.src="http://getbootstrap.com/";
```

```
}
```

Свойство ***length*** возвращает количество фреймов (iframe) в текущем окне. Это свойство очень часто используется совместно со свойством frames. Свойство length доступно только дли чтения.

Например, получить количество фреймов (iframe) в текущем окне:

```
var ifremesCount = window.length;
```

Свойство frames возвращает массив фреймов (объектов window), расположенных а текущем окне. Доступ к фрейму (объекту window) осуществляется по индексу (отсчёт индексов начинаются с 0). Для определения количества фреймов в текущем окне или фрейме используйте свойство length.

Например, изменить URL первого фрейма (т.е. фрейма с индексом 0), который расположен а текущем окне:

```
window.frames[0].location="http://getbootstrap.com/";
```

Например, изменить URL всех фреймов расположенных в текущем окне на "http://getbootstrap.com/":

```
var frames = window.frames;
for (i=0;i<iframes.length;i++) {
 iframes[i].location="http://getbootstrap.com/";
}
```

Свойство ***parent*** возвращает родительский объект window по отношению к текущему объекту window. Данное свойство доступно только для чтения.

Например, установить задний фон родительскому объекту window:

```
window.parent.document.body.style.backgroundColor = "red";
```

Свойство ***top*** возвращает основное окно (вкладку).

Например, определим, является ли данное окно основным окном:

```
function checkWindow() {
  if (window.top != window.self){
    document.getElementById("checkWindow").innerHTML = "Это не основное окно (вкладка)";
  }
  else{
    document.getElementById("checkWindow").innerHTML = "Это основное окно";
  }
}
```

Свойство ***self*** возвращает текущее окно. Данное свойство обычно используется в операциях сравнения и доступно только для чтения.

Рассмотрим наиболее сложный пример, в котором продемонстрируем использование всех свойств объекта window для работы с фреймами:

```
<!--index.html-->
<script>
  function changeBackgroundFrames() {
    var iframes = window.frames;
    alert(iframes.length);
```

```
   for (i=0;i<iframes.length;i++) {
     iframes[i].document.body.style.background = "green";
   }
  }
</script>
...
<h1>Свойства объекта window для работы с фреймами</h1>
<p id="test"></p>
<br />
<a href="javascript:changeBackgroundFrames()">Изменить задний фон у фреймов</a>
<br />
<iframe src="index1.html"></iframe>
<iframe src="index2.html" width="500" height="400"></iframe>
<iframe src="index3.html"></iframe>

<!--index1.html-->
<h2>Фрейм 1</h2>

<!--index2.html-->
<h2>Фрейм 2</h2>
<p id="test"></p>
<iframe src="index4.html"></iframe>
<iframe src="index5.html"></iframe>

<!--index3.html-->
<h2>Фрейм 3</h2>

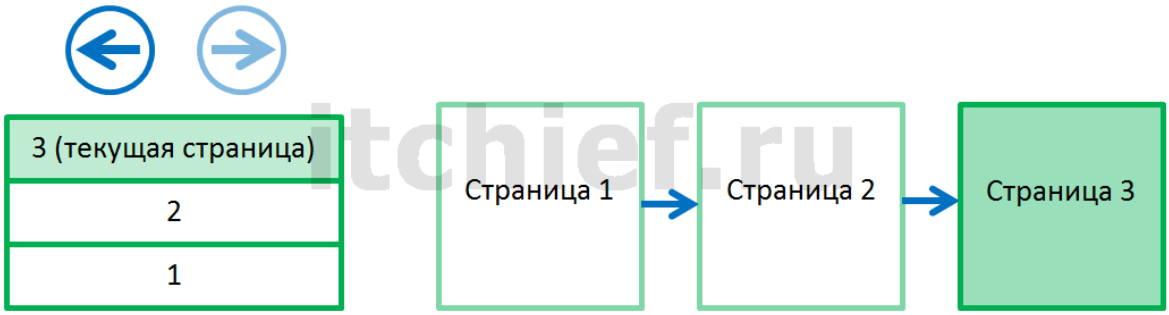
<!--index4.html-->
<script>
  function checkFrame() {
    var iframe = window.frameElement;
    //Вывести сообщение
    if (iframe)
      alert("Это фрейм");
    else alert("Это не фрейм");
  }
  function writeTopWindow() {
    window.top.document.getElementById("test").innerHTML = "Привет из 4 фрейма";
  }
  function writeParentWindow() {
    window.parent.document.getElementById("test").innerHTML = "Привет из 4 фрейма";
  }
</script>
<h2>Фрейм 4</h2>
<a href="javascript:checkFrame()">Это фрейм?</a>
<br />
<a href="javascript:writeTopWindow()">Вывести сообщение в основное окно</a>
<br />
<a href="javascript:writeParentWindow()">Вывести сообщение в родительское окно</a>
```

```
<!--index5.html-->
<h2>Фрейм 5</h2>
```

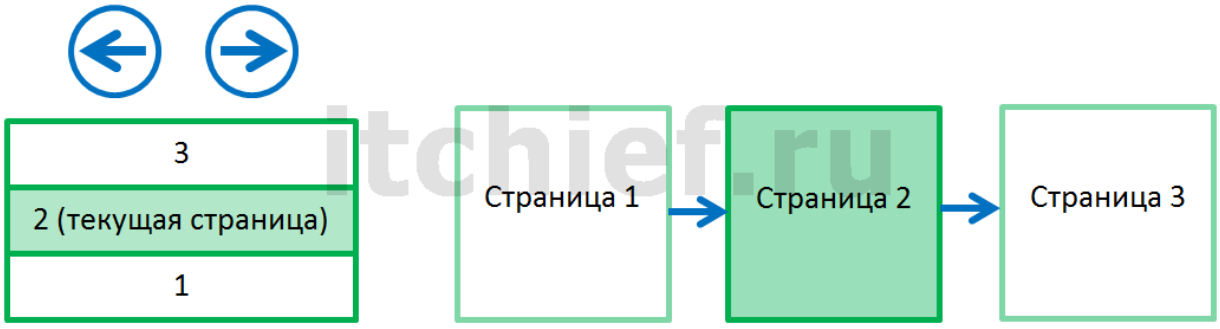
Объект history

Объект ***history*** отвечает за историю переходов, которые пользователь совершил в переделах одного окна или вкладки браузера. Эти переходы браузер сохраняет в сессию истории текущего окна (вкладки) и пользователь с помощи кнопки "Назад" всегда может вернуться на предыдущие страницы. Кроме кнопки "Назад" пользователю доступна ещё кнопка "Вперёд", которая предназначена для того, чтобы пользователь мог вернуться обратно на страницы, с которых он ушел с помощью кнопки "Назад". Таким образом, объект history отвечает за 2 эти кнопки, и позволять не только имитировать их нажатие, но добавлять и изменять записи в истории, перехватывать события при нажатии на эти кнопки и др.

Например, в некотором окне (вкладке) браузера пользователь изначально открыл страницу 1. Просмотрев эту страницу, пользователь нажал в ней на некоторую ссылку и перешёл на страницу 2. На второй странице пользователь снова нажал на ссылку и перешёл на страницу 3. В результате сессия истории для этого окна (вкладки) будет состоять из 3 элементов.



На текущий момент для пользователя доступна кнопка "Назад". При нажатии на эту кнопку пользователь может вернуться на страницу 2.



В этот момент времени для пользователя становится доступны кнопки "Назад" и "Вперёд". При нажатии на кнопку "Назад" пользователь может вернуться на страницу 1, а при нажатии на кнопку "Вперёд" - на страницу 3. Таким образом, пользователь с помощью кнопок "Назад" и "Вперёд" может перемещаться по истории внутри этого окна (вкладки).

Каждый элемент в сессии состоит из:

- **data** - это некоторые данные, которые можно связать с определённым элементом истории. В основном они используются для того чтобы восстановить состояние динамической страницы

при переходе к ней с помощью кнопки "Назад" или "Вперёд". Т.е. сначала Вы сохраняете состояние страницы, а точнее динамических её блоков, которые загружаются с помощью технологии AJAX, в data. А потом, когда пользователь переходит к ней с помощью кнопки "Назад" или "Вперёд", можно восстановить состояние этой страницы, а точнее этих динамических блоков (т.к. одного URL недостаточно) с помощью информации, сохранённой в data.

- **title** - название пункта истории. В настоящее время большинство браузеров не поддерживают возможность установления названия пункта с помощью JavaScript. Браузеры в качестве названия пункта истории используют название страницы, т.е. текст, расположенный между открывающим <titie> и закрывающим </title> тегами HTML страницы.
- **url** - URL адрес страницы.

Свойства, методы и события объекта history:

- **Свойство history.length** - возвращает количество элементов в сессии истории текущего окна (вкладки). Другими словами, данное свойство возвращает количество переходов, которые Вы можете выполнить в пределах текущего окна как вперед, так и назад. Данное свойство доступно только для чтения.
- **Свойство history.state** - позволяет получить данные (data) текущего элемента истории. Эти данные можно добавить к элементу истории с помощью методов history.pushState() и history.replaceState().
- **Метод history.go().** Он позволяет переместить пользователя на некоторое количество страниц вперёд или назад по истории. Метод history.go() имеет один обязательный параметр - это число, на которое надо переместить пользователя вверх или вниз по истории. Например, history.go(2) - осуществляет перемещения пользователя на 2 шаг назад, а history.go(-2) на 2 шага вперёд. Если в качестве параметра указать значение 0, то данный метод вызовет перезагрузку страницы. Если указать в качестве значение число, которое превышает количество шагов, которые может совершить пользователь в текущем окне, то ничего не произойдёт.
- **Метод history.back()** - осуществляет перемещение пользователя на одну страницу назад по истории, т.е. он программно 'имитирует" нажатие кнопки "Назад" в браузере. Также метод history.back() можно выполнить с помощью метода history.go(-1).
- **Метод history.forward()** - осуществляет перемещение пользователя на одну страницу вперёд по истории, т.е. он программно "имитирует" нажатие кнопки "Вперёд" в браузере. Также метод history.forward() можно выполнить с помощью метода history.go(1).
- **Метод history.pushState()** - позволяет добавить новую запись в сессию истории текущего окна (вкладки). Добавление записи (элемента) осуществляется в конец сессии истории.

Метод ***history.pushState(data,title[,url])***.

Данный метод принимает 3 параметра:

- **data (обязательный параметр)** - некоторые данные, которые будут связаны с этой записью в сессии истории. Если в данных нет необходимости, то используйте в качестве параметра значение null.
- **title (обязательный параметр)** - название записи в сессии истории.
- **url (необязательный параметр)** - это URL записи в сессии истории. Если URL не указать, то будет использоваться текущий адрес страницы.



Метод ***history.replaceState(data,title[,url]).***

Метод history.replaceState() - позволяет изменить текущий элемент истории.

Данный метод принимает 3 параметра. Назначение параметров этого метода аналогично методу history.pushState().



Событие popstate.

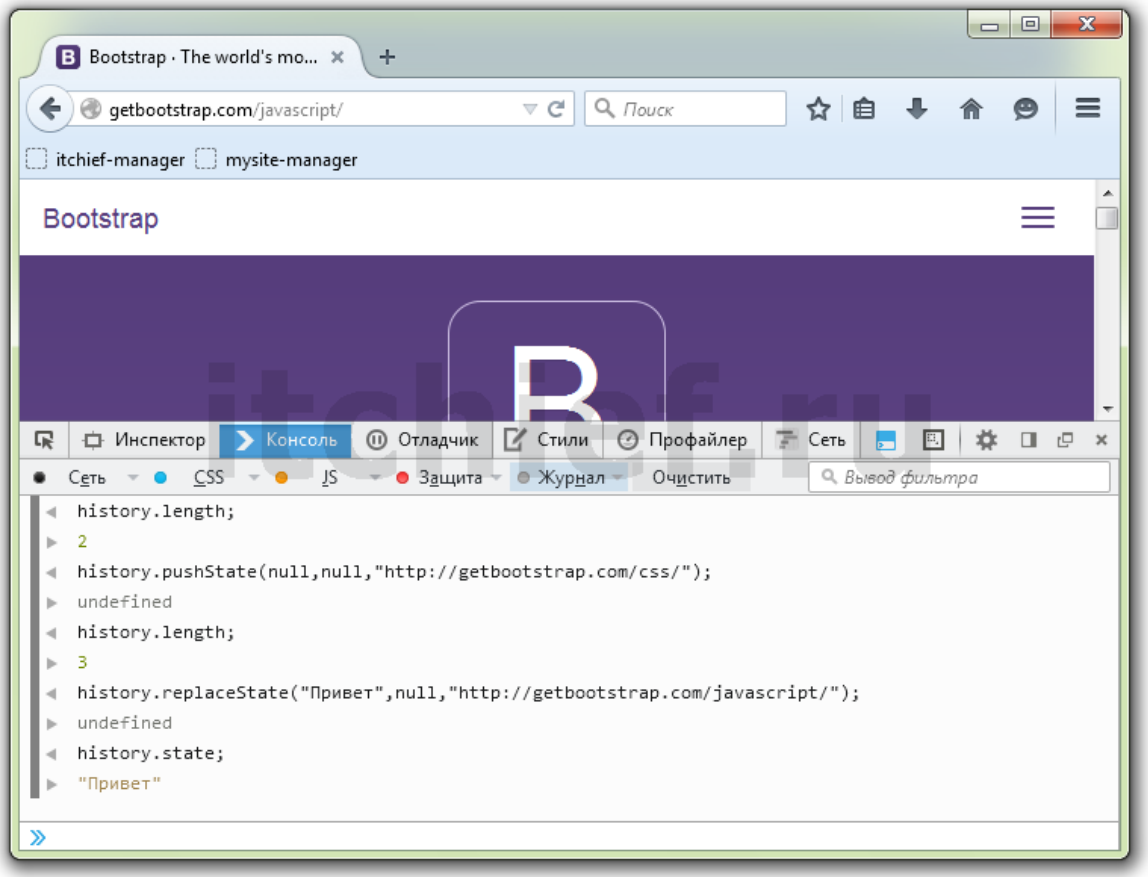
Данное событие вызывает браузер при навигации по истории. Т.е. с помощью его Вы можете обрабатывать события, происходящие в браузере, когда пользователь нажимает на кнопку "Назад" или "Вперёд", или при выполнении методов history.go(), history.back(), history.forward().

```
onpopstate = function(event) {
 //код обработки события popstate
 //например:
 console.log(event.state);
}
window.addEventListener ("popstate", function (e) {
//код обработки события popstate
});
```

В качестве примера рассмотрим работу с объектом history в консоли браузера:

1. Откроем вкладку в браузере и введём в адресной строке http://www.yandex.ru/ и нажмём Enter. В этой же вкладке в адресной строке введём http://getbootstrap.com/ и нажмём опять Enter. Если посмотреть на кнопку "Назад", то он стала активной и позволяет перейти на один шаг назад, т.е. на страницу http://www.yandex.ru/. Кроме этого можно увидеть список элементов истории, если нажать правой кнопкой мыши на кнопку "Назад".
2. Откроем консоль (клавиша F12 в браузере, вкладка "Консоль") и поработаем с историей с помощью JavaScript:
   - Узнаем количество элементов в истории: history.length;
   - Добавим новый элемент в историю: history.pushState(null,null,"http://getbootstrap.com/css/");
   - Перезагрузим страницу с помощью объекта history: history.go(0);
3. Для примера заменим текущий элемент истории и свяжем с ним строку "Привет":
   - history.replaceState("Привет",null,"http://getbootstrap.com/javascript/");

- Получим данные связанные с этим элементом истории: history.state;
- Перейдём на 2 шага назад по истории, т.е. на страницу http://www.yandex.ru/: history.go(-2);



Объект location

Объект ***location*** - это один из дочерних объектов window, который отвечает за адресную строку окна или вкладки браузера. Доступ к данному объекту осуществляется как к свойству объекта window, т.е. через точку.

```
window.location
```

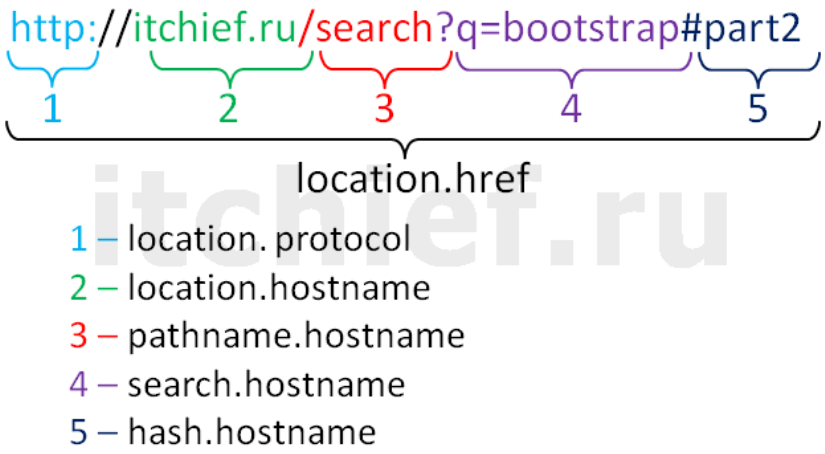
Объект location содержит свойства и методы, с помощью которых Вы можете не только получить текущий адрес страницы (URL или его части: имя хоста, номер порта, протокол и т.д.), но и изменить его.

Свойства объекта location:

- **hash** - устанавливает или возвращает якорную часть (#) URL;
- **host** - устанавливает или возвращает имя хоста и номер порта URL;
- **hostname** - устанавливает или возвращает имя хоста URL;
- **href** - устанавливает или возвращает содержимое URL;
- **origin** - возвращает протокол, имя хоста и номер порта URL;
- **pathname** - устанавливает или возвращает часть URL, содержащей путь;
- **port** - устанавливает или возвращает номер порта URL;
- **protocol** - устанавливает или возвращает протокол URL;

- **search** - устанавливает или возвращает часть URL, содержащей строку с параметрами (?параметр1=значение1&параметр2=значение2&...);

В качестве примера рассмотрим следующую адресную строку (URL):



Примечание: Изменение URL или какой-либо её части с помощью свойств объекта location приводит к немедленному переходу к этому URL в текущем окне, или в том окне или вкладке браузера, для которого этот объект был вызван.

Например, изменить href на http://itchief.ru для текущего окна:

```
window.location.href = "http://itchief.ru";
//для текущего окна "window." можно опустить
location.href = "http://itchief.ru";
//кроме этого свойство href тоже можно опустить, т.к. location == location.href
location = "http://itchief.ru";
```

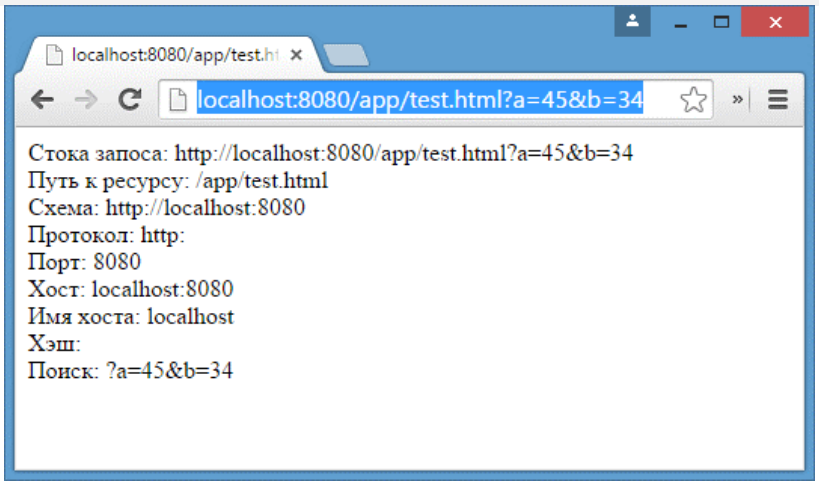
Например, изменить href для окна с именем myWindow:

```
//откроем новое окно с помощью метода JavaScript open()
var myWindow = window.open("","","height=300, width=400");
//изменим location окна, идентификатор которого хранится в myWindow
myWindow.location.href = "http://itchief.ru";
```

Например, пусть есть следующая веб-страница test.html, которая лежит на локальном веб-сервере:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
</head>
<body>
<script>
document.write("Строка запроса: " + location.href + "<br />");
document.write("Путь к ресурсу: " + location.pathname + "<br />");
document.write("Схема: " + location.origin + "<br />");
document.write("Протокол: " + location.protocol + "<br />");
document.write("Порт: " + location.port + "<br />");
document.write("Хост: " + location.host + "<br />");
document.write("Имя хоста: " + location.hostname + "<br />");
document.write("Хэш: " + location.hash + "<br />");
```

```
document.write("Поиск: " + location.search + "<br />");
</script>
</body>
</html>
```



Например, вывести на экран все свойства и методы объекта location и их значения для текущей веб-страницы.

```
<div id="propertiesLocation" class="alert alert-warning"></div>
<script>
 var stringPropertiesLocation = "";
 for (var property in location) {
  stringPropertiesLocation += "Свойство/метод: <strong>" + property + "</strong>. ";
  stringPropertiesLocation += "Значение: <strong>" + location[property] + "</strong> ";
  stringPropertiesLocation += "<strong>Тип: </strong>" + typeof location[property];
  stringPropertiesLocation += "<br />";
 }
 document.getElementById("propertiesLocation").innerHTML = stringPropertiesLocation;
</script>
```

Методы объекта location:

- **assign()** - загружает новый документ в текущее окно (вкладку) браузера или в то окно для которого этот метод был вызван.
- **reload()** - перезагружает текущий документ. Метод reload() имеет один необязательный параметр булевского типа. Если в качестве параметра указать значение true, то страница будет принудительно обновлена с сервера. А если параметр не указывать или использовать в качестве параметра значение false, то браузер может обновить страницу, используя данные своего кэша. Метод reload() "имитирует" нажатие кнопки обновить в браузере.
- **replace()** - заменяет текущий документ с помощью нового документа, URL которого указан в качестве параметра.

Рассмотрим следующие примеры:

При нажатии на ссылку загрузим в 1 фрейм страницу http://nigma.ru/:

```
<script>
 function loadNigma() {
```

```
    window.frames[0].location.assign("http://nigma.ru/");
  }
</script>
...
<!-- Загрузить URL http://yandex.ru в 1 фрейм -->
<a href="javascript:loadNigma()">Загрузить nigma во фрейм</a>
<hr />
<iframe width="500" height="400"></iframe>
```

Откроем новое окно, в котором будем управлять адресной строкой с помощью методов объекта location:

```
<script>
var myWindow;
function myWindowOpen(){
  if (!myWindow)
          myWindow = window.open("https://www.google.com","","width=400, height=500");
}
function myWindowAssign() {
  if (myWindow)
    myWindow.location.assign("http://www.yandex.ru");
}
function myWindowReplace() {
  if (myWindow)
    myWindow.location.replace("http://www.bing.com");
}
</script>
...
<a href="javascript:myWindowOpen()">Открыть окно</a>
<br />
<a href= "javascript:myWindowAssign()">Загрузить URL http://www.yandex.ru</a>
<br />
<a href="javascript:myWindowReplace()">Заменим страницу на URL https://www.bing.com</a>
```

Объект navigator

Объект ***navigator*** — информационный объект, который используется для получения различной информации о браузере, сетевом соединении, операционной системе и т.д.

Вся эта информацию доступна через следующие свойства этого объекта:

- **appCodeName** - возвращает внутреннее "кодовое" имя браузера;
- **appName** - возвращает имя браузера;
- **appVersion** — возвращает информацию о версии браузера;
- **cookieEnabled** - определяет включены ли cookies в браузере;
- **geolocation** - возвращает объект Geolocation, который используется для определения местоположения пользователя;
- **language** - возвращает какой язык используется в браузере;
- **online** - определяет находиться ли браузер в режиме онлайн;
- **platform** - возвращает название платформы, в которой браузер работает;

- **product** - возвращает имя движка, на котором работает браузер;
- **userAgent** - возвращает строку user agent, которая содержит информацию об браузере. Она используется в качестве заголовка, который браузер посылает на сервер.

Кроме свойств объект navigator содержит метод javaEnabled(), который определяет, включена ли поддержка Java в браузере.

Пример использования свойства userAgent, представляющее браузер пользователя:

```
document.write(navigator.userAgent);
```

Данное свойство хранит полную стоку юзер-агента, например,
Mozilla/5.0 (Windows NT 6.3; WOW64)
AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/44.0.2403.157
Safari/537.36

Чтобы вычленить из этой информации непосредственно браузер, можно попробовать найти в этой информации название браузера:

```
var browser, uAgent = navigator.userAgent;
if(uAgent.indexOf("Chrome") > -1) {
    browser = "Google Chrome";
} else if (uAgent.indexOf("Safari") > -1) {
    browser = "Apple Safari";
} else if (uAgent.indexOf("Opera") > -1) {
    browser = "Opera";
} else if (uAgent.indexOf("Firefox") > -1) {
    browser = "Mozilla Firefox";
} else if (uAgent.indexOf("MSIE") > -1) {
    browser = "Microsoft Internet Explorer";
}
document.write(browser);
```

Объект geolocation

Объект ***navigator*** хранит свойство ***geolocation***, с помощью которого можно получить географическое положение пользователя. Для получения положения используется метод ***getCurrentPosition()***. Этот метод принимает два параметра: функцию, которая срабатывает при удачном запуске, и функцию, которая срабатывает при ошибке запроса геоданных:

```
function success(position) {
    var latitude = position.coords.latitude;
    var longitude = position.coords.longitude;
    var altitude = position.coords.altitude;
    var speed = position.coords.speed;

    document.write("Широта: " + latitude + "<br/>");
    document.write("Долгота: " + longitude + "<br/>");
```

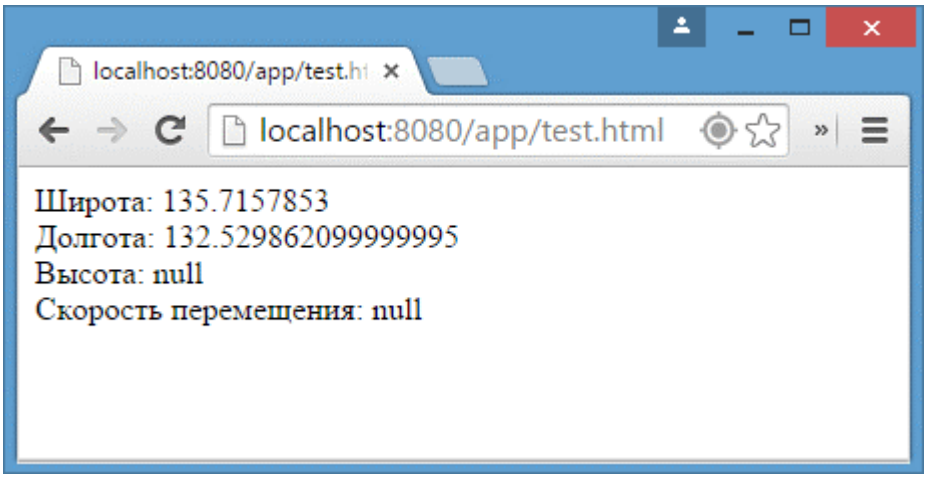
```
        document.write("Высота: " + altitude + "<br/>");
        document.write("Скорость перемещения: " + speed + "<br/>");
    };

    function error(obj) {
        document.write("Ошибка при определении положения");
    };
    navigator.geolocation.getCurrentPosition(success, error);
```

В функцию, которая выполняется при удачном определении геоданных, передается позиция пользователя в виде параметра position. Передаваемый объект имеет вложенный объект coords, с помощью свойство которого можно получить непосредственные координаты пользователя:

- **latitude**: географическая широта
- **longitude**: географическая долгота
- **altitude**: высота
- **speed**: скорость, с которой перемещается пользователь (например, если он идет или перемещается на транспорте)

При этом надо учитывать, что в браузерах действует политика безопасности, которая при обращении к методу geolocation.getCurrentPosition() отображает пользователю сообщение, в котором пользователь может подтвердить отправку географических координат. Если же пользователь откажется, то будет срабатывать функция error().



Таймеры

Для выполнения действий через определенные промежутки времени в объекте window предусмотрены функции таймеров. Есть два типа таймеров: одни выполняются только один раз, а другие постоянно через промежуток времени.

Функция setTimeout

Для одноразового выполнения действий через промежуток времени предназначена функция ***setTimeout()***. Она может принимать два параметра:

```
var timerId = setTimeout(someFunction, period)
```

Параметр period указывает на промежуток, через который будет выполняться функция из параметра someFunction. А в качестве результата функция возвращает id таймера.

```
function timerFunction() {
    document.write("выполнение функции setTimeout");
}
setTimeout(timerFunction, 3000);
```

В данном случае через 3 секунды после загрузки страницы произойдет срабатывание функции timerFunction.

Для остановки таймера применяется функция clearTimeout().

```
function timerFunction() {
    document.write("выполнение функции setTimeout");
}
var timerId = setTimeout(timerFunction, 3000);
clearTimeout(timerId);
```

Функция setInterval

Функции ***setInterval()*** и ***clearInterval()*** работают аналогично функциям setTimeout() и clearTimeout() с той лишь разницей, что setInterval() постоянно выполняет определенную функцию через промежуток времени.

Например, напишем небольшую программу для вывода текущего времени:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
<div id="time"></div>
<script>
function updateTime() {
    document.getElementById("time").innerHTML = new Date().toTimeString();
}
setInterval(updateTime, 1000);
</script>
</body>
</html>
```

Здесь через каждую секунду (1000 миллисекунд) вызывается функция updateTime(), которая обновляет содержимое поля , устанавливая в качестве его кода html текущее время.

requestAnimationFrame()

Метод ***requestAnimationFrame()*** действует аналогично setInterval() за тем исключением, что он больше заточен под анимации, работу с графикой и имеет ряд оптимизаций, которые улучшают его производительность.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    #rect {
        margin: 100px;
        width: 100px;
        height: 100px;
        background: #50c878;
    }
    </style>
</head>
<body>
<div id="rect"></div>
<script>
var square = document.getElementById("rect");
var angle = 0;
function rotate() {
    angle = (angle + 2)%360;
    square.style.transform = "rotate(" + angle + "deg)";
    window.requestAnimationFrame(rotate);
}
var id = window.requestAnimationFrame(rotate);
</script>
</body>
</html>
```

В метод window.requestAnimationFrame() передается функция, которая будет вызываться определенное количество раз (обычно 60) в секунду. В данном случае в этот метод передается функция rotate, которая изменяет угол поворота блока на странице и затем обращается опять же к методу window.requestAnimationFrame(rotate).

В качестве возвращаемого результата метод window.requestAnimationFrame() возвращает уникальный id, который может потом использоваться для остановки анимации:

```
window.cancelAnimationFrame(id);
```

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

1. Стандарт HTML https://html.spec.whatwg.org/multipage/window-object.html#the-window-object
2. Дакетт Джон. HTML и CSS. Разработка и дизайн веб-сайтов. Пер. с англ. – Эксмо, 2017. – 480 с.
3. Прасти Н. Введение в ECMAScript 6. Изд. ДМК, 2016. – 176 с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Дайте определение объектной модели браузера.
2. Что составляет объектную модель браузера?
3. Как можно обратиться к элементам объекта window?
4. Через какие диалоговые окна объекта window можно взаимодействовать с пользователем?
5. Какие параметры указываются при открытии нового окна браузера?
6. Как можно отследить потерю фокуса окном браузера?
7. Можно ли средствами JavaScript перемещать и изменять размеры окна браузера?
8. Какие свойства можно использовать для создания собственного элемента полосы прокрутки?
9. Дайте определение фрейму. Как создать фрейм в окне браузера?
10. Каким образом происходит управление фреймами?
11. Какую полезную информацию предоставляет объект History?
12. Каким образом можно выяснить месторасположение пользователя?